AI HEALTH TRACKER

Complete End-to-End Documentation

A comprehensive guide on system architecture, database layer,

rule-based AI recommendation engines, full REST APIs, and setup walkthrough.

Version: 2.0 (Complete Backend Build)

Created for Vaibhav's AI Health Tracking Project

1. System Introduction & Architecture

The AI Health Tracking System is a modern, high-performance, and resilient full-stack application built to monitor, analyze, and optimize user health. Users can log daily vitals (weight, height, age, sleep, calories, exercise, resting heart rate, and water intake), which are processed through a rule-based AI engine to generate detailed fitness levels, caloric requirements, health risk scores, and actionable workouts, diets, and sleep plans.

Decoupled Full-Stack Architecture

The system utilizes a decoupled single-page application (SPA) architecture:
- Frontend: React.js (built with Vite) for a highly interactive, responsive, and styled UI.
- Backend: Node.js + Express.js gateway API for high-throughput authentication, validation, and medical metric calculations.
- Persistence Layer: A hybrid resilient database engine supporting both remote MongoDB Atlas and a local file-based database fallback (db.json).

2. Directory Structure Map

The project files are structured as follows:

```
ai_health_tracking/
??? backend/
?   ??? .env                          # Port, MongoDB URI, and JWT Secret
?   ??? database.js                   # Database models and fallback persistence engine
?   ??? db.json                       # JSON database file used in offline fallback
?   ??? server.js                     # Express router, auth, and calculations
?   ??? package.json                  # Server runtime dependencies
??? client/
    ??? .env                          # VITE_API_URL configuration
    ??? package.json                  # Client-side dependencies
    ??? src/
        ??? components/               # Modular React UI Elements
        ??? context/                  # AuthContext for session management
        ??? pages/                    # App dashboard pages (Home, Login, Dashboard)
        ??? services/                 # api.js Axios configuration with mock mode
```

3. Database Architecture & Resilient Fallback

One of the system's core features is its dynamic resilience model. When database.js initializes, it attempts to establish a connection to the remote MongoDB database using Mongoose. If this connection fails (e.g. in offline development environments), it logs a warning and falls back to a custom Object Document Mapper (ODM) built over a local file (db.json).

Mongoose & Custom Mock Database Schemas

We support four main collections/tables across both database engines:
1. Users: Stores demographic profiles, credentials, and custom targets.
2. HealthRecords: Logs chronological user vital signs.
3. Predictions: Saves health scores, BMI calculations, and risk evaluations.
4. Recommendations: Caches lists of custom workouts, nutrition tips, and sleep metrics.

Collection	Primary Fields	Description
Users	name, email, password, targetCalories, targetSleep	Demographic user info
HealthRecords	user_id, weight, height, sleep_hours, calories_consumed	Vitals data logs
Predictions	user_id, record_id, bmi, health_score, risk_level	Calculated AI metrics
Recommendations	user_id, record_id, workout[], diet[], sleep[]	Custom action directives

4. Backend REST API Reference

All endpoints require a JSON Web Token (JWT) sent in the HTTP Request Header, except for authentication signup and login. Format: Authorization: Bearer <TOKEN>.

Authentication Endpoints

- POST /auth/signup: Registers a new user. Returns user object and JWT access token.
- POST /auth/login: Authenticates credentials. Returns user object and JWT access token.
- GET /auth/me: Fetches current authenticated user profile details.
- PUT /auth/profile: Updates demographic fields and user daily goals.

Health & Tracking Endpoints

- POST /health/record: Submits daily vitals. Validates inputs, saves to MongoDB/local db, calculates diagnostics, and returns full health evaluation metrics.
- GET /health/history: Retrieves all logs sorted by date descending for charts and table grids.
- DELETE /health/record/:id: Deletes a specific vitals entry.
- GET /predict/latest: Fetches predictions (BMI, health score, sleep quality) from the latest log.
- GET /recommend/latest: Retrieves the newest generated workout, diet, and sleep checklists.
- GET /analytics/daily | weekly | monthly: Returns consolidated summaries (averages and totals) for the respective date ranges.

5. AI Diagnostics & Calculations Engine

The server computes clinical health indexes dynamically when records are logged:

Biometric Equations

- Body Mass Index (BMI): weight (kg) / (height (m))^2
- Basal Metabolic Rate (BMR) - Mifflin-St Jeor:
  - Male: 10 * weight + 6.25 * height - 5 * age + 5
  - Female: 10 * weight + 6.25 * height - 5 * age - 161
- Daily Calorie Needs: BMR * 1.2 (sedentary) + exercise_minutes * 5

Health Score Deductions Index

Starts at 100 points, subtracting for critical vital variances:
- Abnormal BMI (Underweight / Overweight): -10 points
- Obese BMI: -20 points
- Sleep Deficit (<7h or >9h): -15 points
- Anomalous Resting Heart Rate (<60 bpm or >100 bpm): -15 points
- Insufficient Active Time (<30 min daily): -10 points
- Dehydration (<2.0L water daily): -10 points

Rule-Based Recommendation Engine

Custom fitness checklists are generated using diagnostic parameters:
- Overweight/Obesity: Appends '45 min cardio daily', 'strength training 3x/week', '300 kcal daily deficit', and 'increase protein to 120g'.
- High Health Risk: Triggers 'low-intensity only' and flags medical consultations.
- Sleep Deficits: Appends 'consistent bedtime before 11pm', 'no caffeine after 2pm', and 'no screens 1h before bed'.
- Elevated Heart Rate (>90 bpm): Recommends 'focus on low-impact exercise'.

6. Setup & Execution Walkthrough

Follow these steps to run the complete AI Health Tracking system locally:

Environment Settings

Create a .env file in the backend folder:

PORT=8000
MONGO_URI=mongodb://localhost:27017/health_tracking_ai
JWT_SECRET=your-secure-jwt-secret-string-here
JWT_EXPIRATION_HOURS=24

Create a .env file in the client folder:

VITE_API_URL=http://localhost:8000

Running the Project

```
# Step 1: Install and run the backend server
cd backend
npm install
npm start

# Step 2: Install and run the React client
cd ../client
npm install
npm run dev
```

Once running, navigate to http://localhost:5173 to interact with the application. All calculations, database connections, and custom recommendation flows will be handled automatically by the completed Express backend.